



VERIFIZIERTE C-LIBRARY FÜR LOCKFREIE DATENSTRUKTUREN

ENTWICKLUNG DER LOCKFREIEN DATENSTRUKTUREN

Software-Entwicklungspraktikum (SEP)

Sommersemester 2023

Testspezifikation

Auftraggeber

Technische Universität Braunschweig

Institute of Theoretical Computer Science

Prof. Dr. rer. nat. Roland Meyer

IZ 346, i.e. Informatikzentrum, 3rd floor, office 346, Mühlenpfordtstr. 23

D-38106 Braunschweig

Betreuer: Sören van der Wall

Auftragnehmer:

Name	E-Mail-Adresse
Keanu Wirz	k.wirz@tu-braunschweig.de
Nazli Cesur	n.cesur@tu-braunschweig.de
Tristen Vaeckenstedt	t.vaeckenstedt@tu-braunschweig.de
Jacob Müller	jacob.mueller@tu-braunschweig.de
Mohamed Amine Halila	m.halila@tu-braunschweig.de
Omar Zitouni	o.zitouni@tu-braunschweig.de

Braunschweig, 12. Juli 2023

Bearbeiterübersicht

Kapitel	Autoren
1	Omar
2	Mohamed Amine, Keanu
3	Nazli, Omar
3.1	Nazli, Omar
3.2	Nazli, Omar
3.3	Nazli, Omar
4	Tristen, Jacob
4.1	Tristen, Jacob
4.2	Tristen, Jacob
4.3	Tristen, Jacob
5	Keanu
5.1	Keanu
5.2	Keanu
5.3	Keanu
6	Alle

Inhaltsverzeichnis

1	Einleitung	6
2	Testplan	7
2.1	Zu testende Komponenten	8
2.2	Zu testende Funktionen/Merkmale	10
2.3	Nicht zu testende Funktionen	10
2.4	Vorgehen	10
2.4.1	Komponententest	11
2.4.2	Integrationstest	11
2.4.3	Systemtest	11
2.4.4	Abnahmetest	11
2.5	Testumgebung	11
3	Abnahmetest	13
3.1	Zu testende Anforderungen	13
3.2	Testverfahren	14
3.2.1	Testskripte	14
3.3	Testfälle	14
3.3.1	Testfall $\langle T10 \rangle$ - Einfügen in eine leere Datenstruktur	15
3.3.2	Testfall $\langle T20 \rangle$ - Gleichzeitiges Einfügen gleicher Daten	16
3.3.3	Testfall $\langle T30 \rangle$ - Gleichzeitiges Einfügen unterschiedlicher Daten	17
3.3.4	Testfall $\langle T40 \rangle$ - Löschen einzelner Daten	18
3.3.5	Testfall $\langle T50 \rangle$ - Löschen gleicher Daten in mehreren Threads	19
3.3.6	Testfall $\langle T60 \rangle$ - Löschen unterschiedlicher Daten in mehreren Threads	20
3.3.7	Testfall $\langle T70 \rangle$ - Löschen in leerer Datenstruktur	21
3.3.8	Testfall $\langle T80 \rangle$ - UI Test	22
3.3.9	Testfall $\langle T90 \rangle$ - Benchmark Test	23
3.3.10	Testfall $\langle T100 \rangle$ - Search Test	24
4	Integrationstest	25
4.1	Zu testende Komponenten	25
4.2	Testverfahren	25
4.2.1	Testskripte	25

4.3	Testfälle	25
4.3.1	Testfall $\langle T200 \rangle$ - C10 + C20 + C30 + C40	26
4.3.2	Testfall $\langle T300 \rangle$ - Komponente C40 + C50	27
5	Unit-Tests	28
5.1	Zu testende Komponenten	28
5.2	Testverfahren	29
5.2.1	Testskripte	29
5.2.2	Skripte im Kern	29
5.3	Testfälle	30
5.3.1	Testfall $\langle T100 \rangle$ - Set	30
5.3.2	Testfall $\langle T110 \rangle$ - Queue	31
5.3.3	Testfall $\langle T120 \rangle$ - Stack	32
5.3.4	Testfall $\langle T200 \rangle$ - Benchmark	33
5.3.5	Testfall $\langle T300 \rangle$ - Inputwindow	34
5.3.6	Testfall $\langle T301 \rangle$ - Inputwindow	35
5.3.7	Testfall $\langle T310 \rangle$ - Graph	36

Abbildungsverzeichnis

1 Einleitung

Der Zweck von Softwaretests besteht darin, zu überprüfen, ob eine Software die spezifizierten Anforderungen erfüllt und ihre Qualität den Standards entspricht. Durch sorgfältige Evaluierung der Ergebnisse können Softwarefehler aufgedeckt und behoben werden. Die Einbindung von Tests während des gesamten Softwareentwicklungsprozesses stellt sicher, dass das Endprodukt einwandfrei, von höchster Qualität und den spezifischen Anforderungen entsprechend ist. Die Struktur und Spezifikation der Tests basieren auf den Richtlinien des IEEE 829 Standards. Eine Lock-freie Datenstruktur Bibliothek ermöglicht es, dass mehrere parallele Operationen auf gemeinsam genutzten Daten ausgeführt werden können, ohne dass Locks oder andere Synchronisationsmechanismen verwendet werden müssen. Damit dies erfolgreich geschieht, ist es unabdingbar, dass das Produkt vorab gründlich getestet wird.

Deshalb werden wir Unit Tests benutzen, um einzelne Komponenten isoliert von anderen Teilen des Codes zu testen. Hierbei werden kleine Testfälle erstellt, um sicherzustellen, dass die Komponenten wie erwartet funktionieren. Ein Beispiel für die durchgeführten Unit Tests ist das Testen von Enqueue-Operationen auf einer Queue, um sicherzustellen, dass die erwarteten Ergebnisse und Reihenfolgen erzielt werden.

Außerdem werden Benchmarks benutzt, die die Leistungsfähigkeit messen und vergleichen. Hierbei sollten spezifische Szenarien definiert werden, die die lock-freie Datenstruktur ausführen muss, um ihre Leistungsfähigkeit zu messen und zu vergleichen. Beispiele für Benchmarks sind Tests, die die Leistung des Stacks messen, indem sie eine große Anzahl von push- und pop-Operationen in kurzer Zeit ausführen.

Im folgenden Dokument werden die Testabläufe beschrieben, die für die Funktionen unseres Produkts entworfen worden sind.

2 Testplan

Der Testplan ist das zentrale Dokument der Qualitätssicherung und wird daher frühzeitig erstellt. Hier wird Umfang und Vorgehensweise der Qualitätssicherung beschrieben. Außerdem werden Testgegenstände und deren zu testenden Eigenschaften bzw. Funktionen identifiziert. Ferner werden die durchzuführenden Maßnahmen und die dafür verantwortlichen Personen definiert. Falls erforderlich, sollte hier auch auf allgemeine Risiken eingegangen werden.

2.1 Zu testende Komponenten

- **Set <C10>** : Set ist eine Datenstruktur, deren Elemente sortiert sind und nur auf einmal vorkommen dürfen. Hierbei muss getestet werden, ob ein insertender Key schon vorkommt und er an dem richtigen Ort in Set gesetzt wird. Bei der Delete Operation soll überprüft werden, ob die Elemente nach dem löschen in der richtigen Folge bleiben. Zusätzlich muss getestet werden, ob die erwähnten Set Operationen mit mehreren Threads bugfrei durchgeführt werden können, ohne Locks zu benutzen. Die Software liegt als C Bibliothek vor. Es gibt keinen Einfluss auf die Hardware.
- **Queue <C11>** : In der Queue werden die Elemente im FIFO Prinzip hinzugefügt bzw. entfernt. Ähnlich wie bei dem Stack muss getestet werden, ob die Queue diese Operationen mit mehreren Threads bugfrei durchführen kann, ohne Locks zu benutzen. Insbesondere muss getestet werden, ob die Enqueue und Dequeue Funktionen auch mit Interleaving der Threads die zu erwartenden Ergebnisse liefern. Die Software liegt als C Bibliothek vor. Es gibt keinen Einfluss auf die Hardware.
- **Stack <C12>** : Stack ist eine Datenstruktur, wo man im LIFO Prinzip die Elemente hinzufügen bzw. entfernen kann. Hierbei muss getestet werden, ob der Stack diese Operationen mit mehreren Threads bugfrei durchführen kann, ohne Locks zu benutzen. Insbesondere muss getestet werden, ob die Push und Pop Funktionen auch mit Interleaving der Threads die zu erwartenden Ergebnisse liefern. Die Software liegt als C Bibliothek vor. Es gibt keinen Einfluss auf die Hardware.
- **Inputwindow <C20>** : Das Inputwindow dient dazu dem User zu ermöglichen die gewünschten Werte übersichtlich und einfach an den Benchmark weiterzugeben. Es besitzt daher mehrere Textfelder die mit Werten gefüllt werden können und bei einem Knopfdruck wird dann der Benchmark gestartet. Diese Komponente muss auf Korrekte funktionsweise mit gültigen als auch ungültigen Eingaben getestet werden. Dies heißt im genaueren, ob mit gewünschten Werten der Benchmark auch tatsächlich ausgeführt wird oder ob bei ungültigen Werten eine Warnung mit Erklärung ausgegeben wird.
- **Benchmark <C30>** : Der Benchmark nimmt die gewünschten Werte entgegen und misst mit ihnen die Leistungen der Einzelnen Operationen sowie die die Gesamtdauer. Diese werden dann in eine .csv Datei gespeichert die dann weiterverwendet wird. Um eine korrekte Funktionsweise zu garantieren muss der Benchmark auf Korrektheit der Werte überprüft und getestet werden.
- **Graph <C31>** : Der Graph zeigt den Effizienzunterschied zwischen lock-free und lock-base Datenstrukturen. Es wird überprüft ob der Graph die .csv Datei richtig auslesen und

die Werte vernünftig weiterverarbeiten kann. Unter anderem muss beobachtet werden, ob der Graph anschaulich und lesbar ist.

2.2 Zu testende Funktionen/Merkmale

- **F10** Pushen in Stack
- **F20** Pop aus Stack
- **F30** Enqueue
- **F40** Dequeue
- **F50** Insert in Set
- **F60** Delete in Set
- **F80** Search in Set
- **F90** Benchmark visualisieren
- **F100** Benchmark

2.3 Nicht zu testende Funktionen

- Die Betriebssysteme: Linux, macOS und Windows
- CLion (IDE)
- CMake (Build-Tool)
- C standard libraries

Die aufgezählten Funktionen sind durch Dritte entwickelt und getestet wurden, es ist davon auszugehen, dass sie keine Bugs aufweisen.

2.4 Vorgehen

Die allgemeinen Vorgehensweisen für die einzelnen zu testenden Funktionen und Funktionskombinationen werden hier beschrieben. Die Beschreibung sollte detailliert genug sein, um die Hauptaktivitäten und deren Zeitbedarf abschätzen zu können.

2.4.1 Komponententest

Die in 2.1 beschriebenen Komponenten werden auf ihre korrekte Arbeitsweise getestet. Dies beinhaltet zum einen die Datenstrukturen, den Benchmark als auch die Visualisierung als große Gruppen. Die Datenstrukturen werden auf ihre Korrektheit hinsichtlich des Einfüges/Entfernens getestet, der Benchmark darauf, dass er verwendbare und realistische Werte misst und ausgibt und die Visualisierung darauf, dass sie für den User leicht verwendbar ist und die Werte anschaulich darstellen kann.

2.4.2 Integrationstest

Nachdem die Komponenten isoliert funktionieren, werden die Datenstrukturen in einer Library integriert und parallel getestet. Ziel ist es Fehler zu finden, ob die Datenstrukturen sich gegenseitig beeinträchtigen. Unter anderem ist insbesondere die Kombination aus Benchmark und Datenstruktur zu testen, dies bedeutet ob der Benchmark die Funktionen vernünftig und fehlerfrei verwendet. Außerdem muss das Zusammenspiel aus Visualisierung und Benchmark getestet werden, hinsichtlich des Aufrufens des Benchmarks als auch des Auslesens der generierten Werte.

2.4.3 Systemtest

Beim Systemtest sind alle Komponenten integriert und bereits getestet. Jetzt werden letzte Fehler und Warnungen beseitigt. Die Testumgebung soll sich so nah wie möglich an der Anwendungsumgebung des Kunden orientieren. Zudem wird geprüft, wie viel von der Library von außen einsehbar ist und ob dies sich auf die Funktionalität auswirken kann.

2.4.4 Abnahmetest

Wenn alle Testphasen durchlaufen sind und keine Fehler mehr auftreten, wird der Abnahmetest beim Kunden stattfinden. Hier testet der Kunde ob die Library in seiner Systemumgebung funktioniert. Bei dem Test geht es um die Zufriedenheit des Kunden mit dem Produkt.

2.5 Testumgebung

Die genutzte Hardware ist nicht festgelegt, somit ist jeder Desktop-PC und jedes Notebook geeignet. Die genutzten Betriebssysteme sind Linux, macOS und Windows. Bis auf die Betriebssysteme wird die verwendete Software festgelegt, mit CLion als IDE, clang als Compiler, CMake

als Build-Tool und die C standard libraries. Für grafische Benutzeroberflächen (GUIs) wird JavaFX verwendet.

3 Abnahmetest

Abnahme des Produkts durch Kunden.

3.1 Zu testende Anforderungen

In diesem Abschnitt werden alle zu testenden Anforderungen aufgeführt

Nr	Anforderung	Testfälle	Kommentar
1	F10 Pushen in Stack	T10 T20 T30	
2	F20 Poppen aus Stack	T40 T50 T60 T70	
3	F30 Enqueue	T10 T20 T30	
4	F40 Dequeue	T40 T50 T60 T70	
5	F50 Insert in Set	T10 T20 T30	
6	F60 Delete in Set	T40 T50 T60 T70	
7	F80 Search in Set	T10 T20 T30 T40 T50 T60 T70 T100	Search wird in allen Set Funktionen verwendet, die Tests funktionieren nur bei korrekter implementierung dieser Funktion.
8	F90 Benchmark visualisieren	T80	
9	F100 Benchmark	T90	

3.2 Testverfahren

Als Testverfahren werden Black-Box- Tests verwendet. Das Verfahren überprüft, ob die lock freie Datenstrukturen bei einer Eingabe die erwartete Ausgabe liefert ohne Informationen über innere Strukturen oder Source Code zu benötigen, um das korrekte Verhalten der Komponenten zu garantieren.

3.2.1 Testskripte

Es wird keine Testskripte benutzt.

3.3 Testfälle

3.3.1 Testfall $\langle T10 \rangle$ - Einfügen in eine leere Datenstruktur

Ziel

Überprüfen ob Einfügen wie gewünscht funktioniert.

Objekte/Methoden/Funktionen

Je nach Datenstruktur Push **F10**, Insert **F50**, Enqueue **F30**.

Pass/Fail Kriterien

Datenstruktur enthält Objekt an richtiger Stelle und ist in funktionalem Zustand.

Vorbedingung

Es muss eine initialisierte Version der Datenstruktur vorliegen.

Einzelschritte

1. Datenstruktur und Value festlegen. (Beispielsweise List und 3)
2. Funktion ausführen (Hier insert(*list, 3))
3. Überprüfe, ob Daten in Datenstruktur (Node mit Value 3 befindet sich in der Mitte mit return 1)

Ausgabe: 1 für success oder 0 für failed.

Beobachtungen / Log / Umgebung

Beobachtet, ob das Element an der richtigen Stelle und mit richtigen Wert in der Datenstruktur liegt.

Besonderheiten

Als Funktionen sind für Schritt 2 **F10**, **F50** oder **F30** möglich.

Abhängigkeiten

Keine Abhängigkeiten.

3.3.2 Testfall $\langle T20 \rangle$ - Gleichzeitiges Einfügen gleicher Daten

Ziel

Überprüfe, ob Einfügen mit mehreren Threads möglich ist.

Objekte/Methoden/Funktionen

Je nach Datenstruktur Push **F10**, Insert **F50**, Enqueue **F30**.

Pass/Fail Kriterien

Datenstruktur enthält Objekt an richtiger Stelle und ist in funktionalem Zustand.

Vorbedingung Mehrere Threads sind initialisiert und es muss eine initialisierte Version der Datenstruktur vorliegen.

Einzelschritte

1. Datenstruktur und Value festlegen. (Beispielsweise List und 3)
2. Funktion in beiden Threads ausführen (Hier insert(*list, 3))
3. Überprüfe, ob Daten in Datenstruktur nur einmal vorhanden. (Node mit Value 3 befindet sich in der Mitte mit return 1 in einem Thread und 0 in den anderen.)

Ausgabe: Einmal 1 ansonsten 0.

Beobachtungen / Log / Umgebung

Besonderheiten

Basiert auf Testcase **T10** aber mit mehreren Threads.

Abhängigkeiten

T10 muss erfolgreich sein.

3.3.3 Testfall $\langle T30 \rangle$ - Gleichzeitiges Einfügen unterschiedlicher Daten

Ziel

Überprüfe, ob Einfügen mit mehreren Threads möglich ist.

Objekte/Methoden/Funktionen

Je nach Datenstruktur Push **F10**, Insert **F50**, Enqueue **F30**.

Pass/Fail Kriterien

Datenstruktur enthält Objekte an richtiger Stelle und ist in funktionalem Zustand.

Vorbedingung

Mehrere Threads sind initialisiert und es muss eine initialisierte Version der Datenstruktur vorliegen.

Einzelschritte

1. Datenstruktur und Value festlegen. (Beispielsweise List, 3 und 5)
2. Funktion in beiden Threads ausführen (Hier `insert(*list, 3)` und `insert(*list, 5)`)
3. Überprüfe, ob Daten in Datenstruktur nur vorhanden. (Node mit Value 3 befindet sich in der Mitte vor Node mit Key 5 mit return 1 in beiden Threads.)

Ausgabe: 1

Beobachtungen / Log / Umgebung

Wird überprüft, ob Threads synchronisiert die Elemente einfügt.

Besonderheiten

Basiert auf Testcase **T20** aber mit mehreren Daten.

Abhängigkeiten

T20 muss erfolgreich sein.

3.3.4 Testfall $\langle T40 \rangle$ - Löschen einzelner Daten

Ziel

Überprüfe, ob Entfernen möglich ist.

Objekte/Methoden/Funktionen

Je nach Datenstruktur Pop **F20**, Delete **F60**, Dequeue **F40**.

Pass/Fail Kriterien

Datenstruktur hat das Element nicht und die Datenstruktur wurde dementsprechend richtig geändert.

Vorbedingung

Es muss eine initialisierte Version der Datenstruktur mit Daten vorliegen.

Einzelschritte

1. Datenstruktur und Value festlegen. (Beispielsweise List, 3)
2. Funktion ausführen (Hier Delete(*list, 3))
3. Überprüfe, ob Daten nicht mehr in Datenstruktur vorhanden. (Node mit Value 3 befindet sich nicht mehr in der Datenstruktur.)

Ausgabe: 1

Beobachtungen / Log / Umgebung

Wird überprüft, ob das Element richtig entfernt wird und die Datenstruktur richtig geändert wird.

Besonderheiten

Keine Besonderheiten.

Abhängigkeiten

Keine Abhängigkeiten.

3.3.5 Testfall $\langle T50 \rangle$ - Löschen gleicher Daten in mehreren Threads

Ziel

Überprüfe, ob Entfernen möglich ist.

Objekte/Methoden/Funktionen

Je nach Datenstruktur Pop **F20**, Delete **F60**, Dequeue **F40**.

Pass/Fail Kriterien

Datenstruktur enthält Objekte nicht mehr.

Vorbedingung

Es muss eine initialisierte Version der Datenstruktur mit Daten vorliegen.

Einzelschritte

1. Datenstruktur und Value festlegen. (Beispielsweise List, 3)
2. Funktion in mehreren Threads ausführen (Hier Delete(*list, 3))
3. Überprüfe, ob Daten nicht mehr in Datenstruktur vorhanden. (Node mit Value 3 befindet sich nicht mehr in der Datenstruktur.)

Ausgabe: 1 in einem und 0 in anderen Threads.

Beobachtungen / Log / Umgebung

Das Element wurde richtig gelöscht auch mit mehreren Threads.

Besonderheiten

keine Besonderheiten.

Abhängigkeiten

3.3.4 muss erfolgreich sein. Hängt von **T40** ab.

3.3.6 Testfall $\langle T60 \rangle$ - Löschen unterschiedlicher Daten in mehreren Threads

Ziel

Überprüfe, ob Entfernen möglich ist.

Objekte/Methoden/Funktionen

Je nach Datenstruktur Pop **F20**, Delete **F60**, Dequeue **F40**.

Pass/Fail Kriterien

Datenstruktur enthält Objekte nicht mehr.

Vorbedingung

Es muss eine initialisierte Version der Datenstruktur mit Daten vorliegen.

Einzelschritte

1. Datenstruktur und Value festlegen. (Beispielsweise List, 3, 5)
2. Funktion in mehreren Threads ausführen (Hier Delete(*list, 3) und Delete(*list, 5))
3. Überprüfe, ob Daten nicht mehr in Datenstruktur vorhanden. (Node mit Value 3 und Node mit Key 5 befinden sich nicht mehr in der Datenstruktur.)

Ausgabe: 1 in allen Threads.

Beobachtungen / Log / Umgebung

Beobachtet wird, ob die Löschen unterschiedliche Daten in mehreren Threads richtig stattgefunden ist.

Besonderheiten

T40 muss erfolgreich sein.

Abhängigkeiten

Hängt von **T40** ab.

3.3.7 Testfall $\langle T70 \rangle$ - Löschen in leerer Datenstruktur

Ziel

Überprüfe, ob Entfernen nicht möglich ist.

Objekte/Methoden/Funktionen

Je nach Datenstruktur Pop **F20**, Delete **F60**, Dequeue **F40**.

Pass/Fail Kriterien

Es wird eine Fehlermeldung ausgegeben.

Vorbedingung

Es muss eine initialisierte Version der Datenstruktur ohne Daten vorliegen und die Threads müssen initialisiert sein.

Einzelschritte

1. Datenstruktur und Value festlegen. (Beispielsweise List, 3)
2. Funktion in allen Threads ausführen (Hier Delete(*list, 3))
3. Überprüfe ob Fehlermeldung geworfen wird. (ERROR: list is empty.)

Ausgabe: 0 in allen Threads.

Beobachtungen / Log / Umgebung

Beobachtet wird, ob die Ausgabe 0 ist.

Besonderheiten

Keine Besonderheiten.

Abhängigkeiten

Keine Abhängigkeiten.

3.3.8 Testfall $\langle T80 \rangle$ - UI Test

Ziel

Überprüfe, ob UI Elemente funktionieren.

Objekte/Methoden/Funktionen

F90

Pass/Fail Kriterien

Jedes UI Element lässt sich bedienen.

Vorbedingung

F100 muss korrekt funktionieren.

Einzelschritte

1. Datenstruktur festlegen. (Beispielsweise List)
2. Threadanzahl eintragen. (Beispielsweise 3)
3. Anzahl Zugriffe festlegen. (Beispielsweise 20 Inserts und 15 Deletions)
4. Betätige Start Knopf.
5. Lassen sich alle Schritte problemlos durchführen gelingt der Test.

Ausgabe: Keine Ausgabe.

Beobachtungen / Log / Umgebung

Keine besonderen Logs oder Beobachtungen.

Besonderheiten

Es wird kein Skript verwendet, dieser Test wird von Hand von 2 Testern ausgeführt.

Abhängigkeiten

F100 muss funktionieren.

3.3.9 Testfall $\langle T90 \rangle$ - Benchmark Test

Ziel

Überprüfe, ob Benchmark funktioniert.

Objekte/Methoden/Funktionen

F100

Pass/Fail Kriterien

Benchmark liefert realistische Werte.

Vorbedingung

F10 bis **F90** müssen korrekt funktionieren.

Einzelschritte

1. Starte Benchmark mit Parametern. (Beispielsweise list, 3, 20 und 15)
2. Überprüfe, ob Timer gestartet werden.
3. Überprüfe, ob Mittelwert korrekt ermittelt. (Beispielsweise 1ms 2ms und 3ms sollen 2ms zurückgeben)
4. Überprüfe, ob Werte in akzeptablem Wertebereich liegen (keine negativ oder extrem unrealistische Werte). (Beispielsweise lock-free: 50ms 2ms lock-based: 100ms 6ms)

Ausgabe: Ergebnis des Benchmarks.

Beobachtungen / Log / Umgebung

Die einzelnen Timer werden geloggt.

Besonderheiten

Die Werte müssen von mindestens zwei Testern abgesegnet werden.

Abhängigkeiten

Alle vorherigen Tests müssen erfolgreich sein.

3.3.10 Testfall $\langle T100 \rangle$ - Search Test

Ziel

Überprüfe, ob Search funktioniert.

Objekte/Methoden/Funktionen

F80

Pass/Fail Kriterien

Search liefert korrekte Node.

Vorbedingung

Keine Vorbedingungen.

Einzelschritte

1. Starte Search mit Parametern. (Beispielsweise Search(*list, *leftnode, 3))
2. Überprüfe, ob korrekte Node zurückgeliefert wurde. (In Set(1,2,4) muss Node mit Value 4 returnt werden und leftnode muss auf Node mit Value 2 zeigen.)
3. Überprüfe, ob leftnode und rightnode benachbart sind. (2.next zeigt auf 4)

Ausgabe: Keine besondere Ausgabe.

Beobachtungen / Log / Umgebung

Funktion gibt den richtigen Node in der Datenstruktur

Besonderheiten

Keine Besonderheiten.

Abhängigkeiten

Keine Abhängigkeiten.

4 Integrationstest

In den folgenden Integrationstests stellen wir die korrekte Kommunikation und Zusammenarbeit zwischen den einzelnen Komponenten (4.1) dar. Die aufgeführten Komponenten sind bereits einzeln getestet worden.

Unser Testziel ist eine fehlerfreie Ausführung der GUI, die in Java geschrieben wurde, mit den einzelnen Datenstrukturen und dem Benchmark, die in C geschrieben wurden.

4.1 Zu testende Komponenten

Nr	Komponenten	Testfälle	Kommentar
1	C10 Harris List-Datenstruktur	T200	
2	C20 Queue-Datenstruktur	T200	
3	C30 Set-Datenstruktur	T200	
4	C40 Benchmark	T200, T300	
5	C50 GUI	T300	

4.2 Testverfahren

Als Testverfahren wird das Black-Box-Verfahren verwendet. Dabei wird die Software geprüft, ob bei einer Eingabe die erwartete Ausgabe zurückgegeben wird bzw. die entsprechende Reaktion erfolgt.

4.2.1 Testskripte

Für die Integrationstests werden keine Testskripte verwendet.

4.3 Testfälle

4.3.1 Testfall $\langle T200 \rangle$ - C10 + C20 + C30 + C40

Ziel

Testet, ob die Datenstrukturen in dem Benchmark korrekt ausgeführt und gemessen werden.

Objekte/Methoden/Funktionen

Benchmark/lock_free_benchmark(datastructures, threads, insertion, deletion)

Benchmark/lock_based_benchmark(datastructures, threads, insertion, deletion)

Pass/Fail Kriterien

Pass: Der Test ist erfolgreich, wenn der Wert der zurückgegebenen Werte konstant steigt und die Gesamtzahl der gemessenen Werte mit der Zahl von Einfüge- und Löschoperationen übereinstimmt.

Fail: Der Test schlägt fehl, wenn keine Werte oder die Werte nicht vollständig zurückgegeben werden.

Vorbedingung

Der PC sollte im Hintergrund keine anderen Tasks bearbeiten, sonst könnten die Ergebnisse stark voneinander abweichen.

Einzelschritte

1. Terminal oder IDE öffnen,
2. Benchmark-Datei mit Übergabeparametern starten,
3. Zurückgegebene Werte kontrollieren.

Beobachtungen / Log / Umgebung

Wie unter den Vorbedingungen sollten während der Berechnung keine weiteren Tasks auf dem PC gestartet werden.

Besonderheiten

Keine Besonderheiten.

Abhängigkeiten

....

4.3.2 Testfall $\langle T300 \rangle$ - Komponente C40 + C50

Ziel

Testet, ob die Eingaben dem Benchmark übergeben werden und die gemessenen Daten passend visualisiert werden.

Objekte/Methoden/Funktionen

GUI/Threads[1, ..., 5]
GUI/Datenstruktur[Harris-List, Queue, Set]
GUI/Betriebssystem[Linux-MacOS, Windows]
GUI/Inserts
GUI/Deletions.

Pass/Fail Kriterien

Pass: Der Test ist erfolgreich, wenn der Graph genauso viele Punkte wie die angegebenen Operationen besitzt.

Fail: Der Test schlägt fehl, wenn keine Werte oder die Werte nicht vollständig angezeigt werden.

Vorbedingung

Keine notwendigen Vorbedingungen.

Einzelschritte

1. Terminal oder IDE öffnen,
2. GUI starten,
3. Im GUI-Fenster die Werte für den Benchmark eingeben,
4. Graphen kontrollieren.

Beobachtungen / Log / Umgebung

Die Werte im Graphen sollten nicht gleich sein, um sicherzustellen, dass die Werte neu berechnet werden.

Besonderheiten

Die GUI interagiert später mit dem Nutzer. Deswegen sollten alle Funktionen der GUI problemlos, optisch ansprechend und schnell ausführbar sein.

Abhängigkeiten

....

5 Unit-Tests

In diesem Kapitel soll die Vorgehensweise im Unit-Test beschrieben werden. Die Bearbeitung soll genauso erfolgen, wie bereits in der Abnahmetestspezifikation der Abnahmetest. Achtet bitte darauf, dass die Nummerierung der Testfälle sich nicht mit der aus der Abnahmetestspezifikation überschneidet.

Testziel: Überprüfen ob die Komponenten Fachgerecht und wie gewünscht funktionieren.

5.1 Zu testende Komponenten

Nr	Komponenten	Testfälle	Kommentar
1	<K10> Set	<T100>	Überprüfe ob sich das Set wie gewünscht verhält
2	<K11> Queue	<T110>	Überprüfe ob sich die Queue wie gewünscht verhält
3	<K12> Stack	<T120>	Überprüfe ob sich der Stack wie gewünscht verhält
4	<K20> Benchmark	<T200>	Überprüfe ob sich der Benchmark wie gewünscht verhält
5	<K30> Inputwindow	<T300>	Überprüfe ob das Inputwindow auf valide Eingaben reagiert
6	<K30> Inputwindow	<T301>	Überprüfe ob das Inputwindow angemessen auf falsche Eingaben reagiert
7	<K31> Graph	<T310>	Überprüfe, dass der Graph anschaulich und gut lesbar ist

5.2 Testverfahren

Als Testverfahren nutzen wir Blackbox Tests, mit denen wir feststellen können ob unsere Software die entsprechenden Ergebnisse liefert und auf ungültige Eingaben wie erwünscht reagiert. Dazu Überprüfen wir edge-cases, ungültige Eingaben und gültige Eingaben und bestimmen den gelieferten Leistungsumfang. Unter anderem überprüfen wir auch ob unsere Datenstrukturen konsistent unter Belastung bleiben, das bedeutet ob die gewünschten Werte in gewünschter Reihenfolge eingefügt/entfernt wurden.

5.2.1 Testskripte

Als Testskripte verwenden wir selbstgeschriebene Skripte, die die zu überprüfende Funktionalität ausführt und mit asserts checkt ob die Funktion erfolgreich und fehlerfrei ausgeführt wurde. Diese sehen in ihrer Grundfunktionalität so aus:

5.2.2 Skripte im Kern

0. Treffe vorbereitungen um die Funktion ausführen zu können
1. Führe Funktion aus
2. Überprüfe mithilfe von return-Wert ob Funktion erfolgreich durchgelaufen ist
3. Überprüfe mithilfe von Asserts ob Ergebnis valide ist.

5.3 Testfälle

5.3.1 Testfall $\langle T100 \rangle$ - Set

Ziel Überprüfe wie die Set Funktionen auf gültige Eingaben reagieren und kontrolliere ob das Set Konsistent bleibt.

Objekte/Methoden/Funktionen Genutzt wird ein initialisiertes Set auf dem die Funktionen Insert und Delete von mehreren Threads zeitgleich ausgeführt werden.

Pass/Fail Kriterien Pass: Alle Operationen sind ohne Fehler durchgeführt worden und die Liste befindet sich in einem konsistentem Zustand. Fail: Operationen brechen ab oder Liste befindet sich nach erfolgreicher durchführung in inkonsistentem Zustand.

Vorbedingung Eine Initialisiertes Set sowie initialisierte threads müssen vorliegen die das Testskript ausführen.

Einzelschritte 1. Erstellen und Initialisieren des Sets

2. Erstellen und initialisieren der Threads

3. Threads führen sowohl Insert als auch Deletion parallel aus.

4. Überprüfe Liste auf ihre konsistenz

Beobachtungen / Log / Umgebung Beobachte ob die Liste am ende nur eingebene Keys enthält. Liste lässt sich in eine Log Datei speichern um die Überprüfung zu erleichtern.

Besonderheiten keine Besonderheiten

Abhängigkeiten keine Abhängigkeiten

5.3.2 Testfall $\langle T110 \rangle$ - Queue

Ziel Überprüfe wie die Queue Funktionen auf gültige Eingaben reagieren und kontrolliere ob die Queue Konsistent bleibt.

Objekte/Methoden/Funktionen Genutzt wird eine initialisierte Queue auf der die Funktionen enqueue und dequeue von mehreren Threads zeitgleich ausgeführt werden.

Pass/Fail Kriterien Pass: Alle Operationen sind ohne Fehler durchgeführt worden und die Queue befindet sich in einem konsistentem Zustand. Fail: Operationen brechen ab oder Queue befindet sich nach erfolgreicher durchführung in inkonsistentem Zustand.

Vorbedingung Eine Initialisiertes Queue sowie initialisierte Threads müssen vorliegen die das Testskript ausführen.

Einzelschritte 1. Erstellen und Initialisieren der Queue

2. Erstellen und initialisieren der Threads

3. Threads führen sowohl enqueue als auch dequeue parallel aus.

4. Überprüfe Queue auf ihre konsistenz

Beobachtungen / Log / Umgebung Beobachte ob die Queue am ende nur eingebene Keys enthält. Queue lässt sich in eine Log Datei speichern um die Überprüfung zu erleichtern.

Besonderheiten keine Besonderheiten

Abhängigkeiten keine Besonderheiten

5.3.3 Testfall $\langle T_{120} \rangle$ - Stack

Ziel Überprüfe wie die Stack Funktionen auf gültige Eingaben reagieren und kontrolliere ob der Stack Konsistent bleibt.

Objekte/Methoden/Funktionen Genutzt wird ein initialisierter Stack auf der die Funktionen push und pop von mehreren Threads zeitgleich ausgeführt werden.

Pass/Fail Kriterien Pass: Alle Operationen sind ohne Fehler durchgeführt worden und der Stack befindet sich in einem konsistentem Zustand. Fail: Operationen brechen ab oder Stack befindet sich nach erfolgreicher durchführung in inkonsistentem Zustand.

Vorbedingung Ein Initialisierter Stack sowie initialisierte Threads müssen vorliegen die das Testskript ausführen.

Einzelschritte 1. Erstellen und Initialisieren des Stacks

2. Erstellen und initialisieren der Threads

3. Threads führen sowohl push als auch pop parallel aus.

4. Überprüfe Queue auf ihre konsistenz

Beobachtungen / Log / Umgebung Beobachte ob der Stack am ende nur eingebene Keys enthält. Stack lässt sich in eine Log Datei speichern um die Überprüfung zu erleichtern.

Besonderheiten keine Besonderheiten

Abhängigkeiten keine Besonderheiten

5.3.4 Testfall $\langle T200 \rangle$ - Benchmark

Ziel Überprüfe wie die Benchmark Funktionen auf gültige Eingaben reagieren und kontrolliere ob Ergebnisse realistisch sind.

Objekte/Methoden/Funktionen Genutzt werden die Benchmark funktionen Benchmark und ThreadBenchmark, die Mehrere Threads erstellen die dann die Performance der Operationen messen.

Pass/Fail Kriterien Pass: Alle Operationen sind ohne Fehler durchgeführt worden und die Ergebnisse liegen in einem realistischen Rahmen. Fail: Operationen brechen ab oder Ergebnisse sind unrealistisch.

Vorbedingung Keine Vorbedingungen nötig, Benchmark Funktionen erledigen alles nötige.

Einzelschritte 1. Starten des Benchmarks mit entsprechenden Daten

2. Erstellen und initialisieren der Threads

3. Threads führen sowohl Einfüge- als auch Entfernen-Operationen parallel aus.

4. Überprüfe Ergebnis auf korrektheit.

Beobachtungen / Log / Umgebung Benchmark Ergebnisse liegen in einer .csv Datei vor.

Besonderheiten keine Besonderheiten

Abhängigkeiten Abhängig von T100 - T120.

5.3.5 Testfall $\langle T300 \rangle$ - Inputwindow

Ziel Überprüfe wie die Inputwindow Funktionen auf gültige Eingaben reagieren und kontrolliere ob Benchmark wie gewünscht ausgeführt wird.

Objekte/Methoden/Funktionen Genutzt werden die JFrame Funktionen.

Pass/Fail Kriterien Pass: Alle Werte lassen sich problemfrei eintragen und Benchmark wird wie gewünscht gestartet. Fail: Benchmark wird nicht gestartet oder bei der Eingabe gab es ein Problem.

Vorbedingung Keine Vorbedingungen nötig.

Einzelschritte 1. Starten des Inputwindow

2. Eintragen der Werte

3. Betätigen des Start Knopfes

4. Überprüfe ob Benchmark gestartet wird und ein valides Ergebnis auftritt.

Beobachtungen / Log / Umgebung Beobachte ob Benchmark Prozess gestartet wird.

Besonderheiten keine Besonderheiten

Abhängigkeiten Abhängig von T200.

5.3.6 Testfall $\langle T301 \rangle$ - Inputwindow

Ziel Überprüfe wie die Inputwindow Funktionen auf ungültige Eingaben reagieren und kontrolliere ob dem User die Problemstelle wie gewünscht mitgeteilt wird.

Objekte/Methoden/Funktionen Genutzt werden die JFrame Funktionen.

Pass/Fail Kriterien Pass: Alle Werte lassen sich problemfrei eintragen und beim starten tritt eine Fehlermeldung mit Beschreibung auf. Fail: Benchmark wird mit ungültigen Werten und ohne Fehler gestartet.

Vorbedingung Keine Vorbedingungen nötig.

Einzelschritte 1. Starten des Inputwindow

2. Eintragen der Werte
3. Betätigen des Start Knopfes
4. Überprüfe ob Fehlermeldung auftritt.

Beobachtungen / Log / Umgebung Beobachte ob die Fehlermeldung die Problemstelle ausreichend beschreibt. Beispielsweise "AC23!" ist keine gültige Zahl für die Anzahl der Inserts!".

Besonderheiten keine Besonderheiten

Abhängigkeiten keine Abhängigkeiten

5.3.7 Testfall $\langle T310 \rangle$ - Graph

Ziel Überprüfe wie der Graph auf gültige Eingaben reagiert und kontrolliere ob dem User die Werte anschaulich visualisiert werden.

Objekte/Methoden/Funktionen Genutzt werden die JavaFX Funktionen.

Pass/Fail Kriterien Pass: Alle Werte lassen sich problemfrei auslesen und in einem Graph abbilden. Fail: Fehler beim Abbilden oder Graph ist nicht anschaulich.

Vorbedingung Keine Vorbedingungen nötig.

Einzelschritte 1. Starten der Graph-Klasse

2. Graph liest die Werte aus .csv Datei aus
3. Graph bildet Ergebnisse ab
4. Überprüfe ob Graph anschaulich abbildet.

Beobachtungen / Log / Umgebung Beobachte ob die Skalen beziehungsweise die Werte an den Skalen lesbar und verwendbar sind.

Besonderheiten keine Besonderheiten

Abhängigkeiten keine Abhängigkeiten

6 Glossar

Kapitel	Autoren
IEEE	Institute of Electrical and Electronics Engineers
lock-frei	ohne Threadlocks
Threadlock	Ein lock das blockiert, dass andere Threads weiterarbeiten
Thread	Prozess mit getiltem Speicherraum
Unit Tests	Tests einer einzelne Komponente
Benchmark	Programm zum bestimmen der Performance
FIFO	First-In-First-Out, Prinzip in dem dass erste eingefügte Element als erstes entfernt wird
LIFO	Last-In-First-Out, Prinzip in dem dass letzte eingefügte Element als erstes entfernt wird
IDE	integrierte Entwicklungsumgebung
CMake	Build tool für Makefiles
Make	Build tool für C Programme
JavaFX	Library für Java Graphenerstellung
Black-box Tests	Tests bei denen nur Input und Output bekannt ist
UI	User-Interface